

RANSAC

1. Overview

1. RANSAC stands for Random Sample Consensus, it is a popular algorithm to robustly fit a model (like a line or plane) when the data contains a large number of outliers.

2. Key steps:

1. Randomly select a small subset of data to form a model.
2. Determine how many points from the entire dataset fit the model (inliers).
3. Update the best model if the new one has more inliers.
4. Adaptively update the number of iterations needed based on the ratio of inliers found.

2. Synthetic Data Generation

1. This function generates a set of N 2D points, consisting of:

1. Inlier points that lie roughly along a line passing through a reference point p in the direction of d .
2. Outlier points scattered randomly within a bounding rectangle $[0, Width] \times [0, Height]$.

2. Steps:

1. Normalize direction vector $\vec{d}$ as:

$$1. \vec{d} = \frac{\vec{d}}{\|\vec{d}\|}$$

2. Partition N into inliers and outliers

$$1. N_{inlier} = \text{floor}(N \times \text{inlier_ratio})$$

$$2. N_{outlier} = N - N_{inlier}$$

3. Generate inliers along a line

1. Define p as base point.

2. select $t \sim \text{Uniform}(-\frac{diag}{2}, \frac{diag}{2})$, where:

$$3. diag = \sqrt{Width^2 + Height^2}.$$

4. Compute each inlier point: $\vec{q} = \vec{p} + t\vec{d}$

5. Add Gaussian noise $\mathcal{N}(0, 1)$ to both coordinates. $q[0] := q[0] + \mathcal{N}(0, 10)$, $q[1] := q[1] + \mathcal{N}(0, 10)$

4. Generate outliers

1. For each of the $N_{outlier}$ points, sample randomly:

$$2. x \sim \text{Uniform}(0, Width)$$

$$3. y \sim \text{Uniform}(0, Height).$$

5. Return Combined array of shape (N, d), d=2.

3. Mathematical explanation of iteration_number(confidence, sample_size, inliers, N)

1. This function computes the number of RANSAC iterations required to ensure, with a given confidence level, that at least one randomly selected subset consists entirely of inliers.

2. Steps:

1. Definitions:

1. Inlier ratio (w): The fraction of points that are inliers.

2. Sample size (s): The minimum number of points needed to estimate the model.

3. Confidence (p): The probability that at least one sample contains only inliers.

4. The probability of picking an inlier at random is:

$$5. w = \frac{\text{inliers}}{N},$$

6. If (inliers=0), then ($w=0$), meaning RANSAC cannot reliably find a correct model. In this case, the function returns (inf) (indicating an unbounded number of iterations might be required).

2. Probability of a good sample

1. The probability of a randomly chosen sample of size (s) consists only of inliers is w^s .

3. Probability of All bad samples

1. If we run (k) iterations, the probability that none of these (k) samples contain only inliers is: $(1 - w^s)^k$.

4. Confidence constraint

1. To ensure a confidence level (p), we require $(1 - w^s)^k \leq 1 - p$.

5. Solving for (k)

1. Taking the natural logarithm of both sides:

$$2. k \ln(1 - w^s) \leq \ln(1 - p).$$

3. Thus $k \geq \frac{\ln(1-p)}{\ln(1-w^s)}$.

4. Since RANSAC requires an integer number of iterations, we take the ceiling: $k = \text{ceil}(\frac{\ln(1-p)}{\ln(1-w^s)})$.

6. Return value

1. If `inliers == 0`, we return (inf) since no inlier-based model can be drawn.
2. If the denominator ($\ln(\dots)$) is zero or extremely small, we also return (inf).
3. Otherwise, we return the ceiling of the above fraction, indicating the required iterations to reach the given confidence level.

4. Implementation:

1. **Initialize the random seed** for reproducibility.
2. **Generate synthetic data** of size 1000, with 50% inliers.
3. **Apply RANSAC** to fit a line.
4. **Refine the best line** using all inliers.

```
# -*- coding: utf-8 -*-
"""
Created on Sun Mar  8 18:22:50 2026

@author: reina
"""

import numpy as np
import matplotlib.pyplot as plt
import random
import math

def init_random(seed=None):
    if seed is not None:
        np.random.seed(seed)
        random.seed(seed)

def generate_data(N, p, direction, inlier_ratio=0.5):
    WIDTH, HEIGHT = 640, 480
    p = np.array(p, dtype=float)
    direction = np.array(direction, dtype=float)
    direction /= np.linalg.norm(direction)

    N_inlier = int(N * inlier_ratio)
    N_outlier = N - N_inlier

    points = []

    diag = np.sqrt(WIDTH**2 + HEIGHT**2)
    for _ in range(N_inlier):
        t = (random.random() * diag) - diag / 2.0
        q = p + t * direction
        q[0] += np.random.normal(0.0, 10.0)
        q[1] += np.random.normal(0.0, 10.0)
        points.append(q)

    for _ in range(N_outlier):
        points.append((random.random() * WIDTH, random.random() * HEIGHT))

    return np.array(points, dtype=float)
```

```

def iteration_number(confidence, sample_size, inliers, N):
    if inliers == 0:
        return float('inf')
    ratio = inliers / float(N)
    top = math.log(1.0 - confidence)
    bottom = math.log(1.0 - ratio**sample_size)
    return math.ceil(top / bottom) if bottom != 0 else float('inf')

def fit_line_two_points(p1, p2):
    x1, y1 = p1
    x2, y2 = p2
    dx, dy = x2 - x1, y2 - y1
    norm = math.sqrt(dx**2 + dy**2)
    if norm < 1e-8:
        return None
    a, b = -dy / norm, dx / norm
    c = -a * x1 - b * y1
    return a, b, c

def distance_point_line(line_params, point):
    a, b, c = line_params
    x, y = point
    return abs(a*x + b*y + c)

def ransac_line(points, max_iterations=2000, sigma=10.0, confidence=0.95):
    best_inliers_count = 0
    best_line, best_inliers_list = None, []

    for _ in range(max_iterations):
        idx1, idx2 = np.random.choice(len(points), 2, replace=False)
        line_params = fit_line_two_points(points[idx1], points[idx2])
        if line_params is None:
            continue

        inliers = [pt for pt in points if distance_point_line(line_params, pt) < sigma]

        if len(inliers) > best_inliers_count:
            best_inliers_count, best_line, best_inliers_list = len(inliers), line_params, inliers
            max_iterations = min(max_iterations, iteration_number(confidence, 2, best_inliers_count, len(points)))
            print(max_iterations)

    return best_line, np.array(best_inliers_list)

if __name__ == '__main__':
    init_random(seed=0)

    N = 1000
    p = (240.0, 320.0)
    alpha = random.random() * 2.0 * np.pi
    dir_ = (np.cos(alpha), np.sin(alpha))
    points = generate_data(N, p, dir_, inlier_ratio=0.5)

    fig, ax = plt.subplots(figsize=(8,6))
    ax.scatter(points[:,0], points[:,1], s=10, c='k', alpha=0.5, label='All Points')

```

```

ax.set_xlim([-50, 690])
ax.set_ylim([-50, 530])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_title('Point cloud for Line Fitting')
ax.legend()
plt.show()

best_line, inliers = ransac_line(points)
print(f"Best line: a={best_line[0]:.3f}, b={best_line[1]:.3f}, c={best_line[2]:.3f}")
print(f"Number of inliers: {len(inliers)} out of {len(points)}")

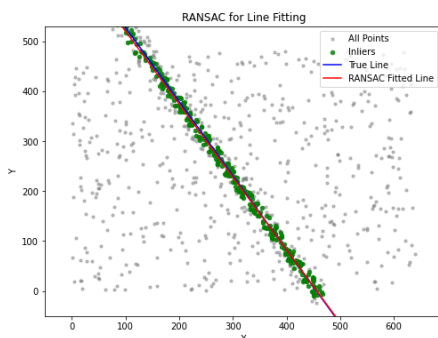
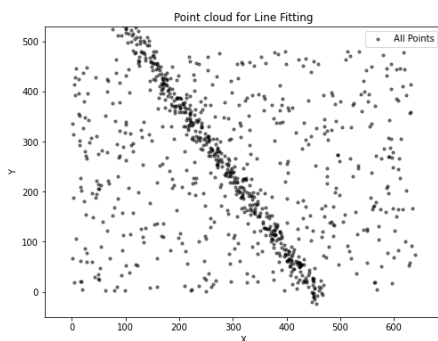
fig, ax = plt.subplots(figsize=(8,6))
ax.scatter(points[:,0], points[:,1], s=10, c='gray', alpha=0.5, label='All Points')
ax.scatter(inliers[:,0], inliers[:,1], s=20, c='green', alpha=0.8, label='Inliers')

p1 = (p[0] - 1000*dir_[0], p[1] - 1000*dir_[1])
p2 = (p[0] + 1000*dir_[0], p[1] + 1000*dir_[1])
ax.plot([p1[0], p2[0]], [p1[1], p2[1]], c='blue', label='True Line')

a, b, c = best_line
x_vals = np.linspace(0, 640, 2)
y_vals = (-a*x_vals - c) / b
ax.plot(x_vals, y_vals, c='red', label='RANSAC Fitted Line')

ax.set_xlim([-50, 690])
ax.set_ylim([-50, 530])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_title('RANSAC for Line Fitting')
ax.legend()
plt.show()

```



References:

1. [3D-CV-Lab3](#), Tarlan Ahadi, Elte, 2025.